code puts "Hello, world!" calls the method puts with the argument "Hello, world!". The puts method then outputs that argument back to the terminal—puts is short for "outPUT String".

On the next line we typed an end-of-file character (which is Ctrl+D on our system), which exited the program and caused what we typed to be evaluated. Using Ruby like this works, but it only shows responses if you explicitly print them out. Also, it's painful if you make a typo, and you can't see what's going on as you type.

Happily, there's a better way to interact with Ruby.

Interactive Ruby, or *irb*, is the tool of choice for executing Ruby interactively. Irb is a complete Ruby shell, with command-line history, line-editing capabilities, and job control. (In fact, it has its own chapter in this book : Chapter 16, Interactive Ruby, on page 277.) You run irb from the command line. Once it starts, type in Ruby code. It will show you the value of each expression as it evaluates it. Exit an irb session by typing exit or Ctrl+D.

Here's a sample session:

```
$ irb
irb(main):001:1* def sum(n1, n2)
irb(main):002:1*   n1 + n2
irb(main):003:0> end
=> :sum
irb(main):004:0> sum(3, 4)
=> 7
irb(main):005:0> sum("cat", "dog")
=> "catdog"
irb(main):006:0> exit
```

In the first three lines of this session, we're defining a method called sum. The act of defining that method returns a value called :sum, which is a Ruby *symbol* matching the name of the method and we'll talk more about symbols and method names later. In line 4 of the input, we're calling the method, first with arguments 3 and 4, returning 7, then on line 5 with arguments "cat" and "dog". In Ruby, adding strings concatenates them, so that line returns the string "catdog". Then we exit on line 6.

If you try this in Ruby 3.1 or higher you'll notice that irb attempts to offer autocompletion of variable names or commands, and also color codes, neither of which is easy to show in a book.

We recommend that you get familiar with irb—it's a great way to explore Ruby concepts and debug your code, and it will make your experience with Ruby more fun.

What About Docker?

If you are using Ruby on a larger project or with a larger team, there's a good chance that Docker is part of your development environment. Docker is a tool that allows you to define and run *containers* A container is a way to package all the dependencies needed to run code, effectively it's a virtual operating system inside your computer. Using Docker, you can simulate a Linux environment no matter what operating system you are running.

A full description of Docker is out of this book's scope, but if you are already familiar with Docker in general, it's worth mentioning that Docker maintains images with different Ruby versions pre-installed, and you can always get to the latest released version with ruby:latest, and you can go straight